Phase 09 — Scaling and Architecture (Complete Notes)

> *What breaks when traffic grows, and the standard playbook for fixing it.*

---

1. Why scaling is needed

One server has limits: CPU cores, RAM, connections, disk IO. Load beyond that → queues → latency climbs → timeouts (504s! — Phase 5) → crash. Two dimensions of trouble:

- **Performance**: more users than capacity.
- **Availability**: one server = one failure away from total outage (deploys, kernel updates, hardware death).

Analogy: a restaurant with one cook. More customers → orders queue → food is late → customers leave. Options: a faster cook (vertical) or more cooks (horizontal) — and more cooks need a head waiter to distribute orders (load balancer) and a shared order board (shared state).

2. Vertical scaling (scale UP)

Bigger machine: 2→16 vCPU, 4→64 GB.

- ✅ Trivial: no code changes, resize the VPS/EC2 and reboot.
- ❌ Hard ceiling; cost grows non-linearly; still ONE machine (no availability gain); resize = downtime.
- **Right choice more often than juniors think**: a $40 server handles enormous load for a well-written app. Exhaust easy vertical scaling before adding architecture.

3. Horizontal scaling (scale OUT)

More machines, traffic split across them:

- ✅ Near-limitless; **high availability** (one dies, others carry on); rolling deploys with zero downtime; can scale automatically.
- ❌ Requires: a load balancer (Phase 10) and — the big one — **stateless applications**.

4. Stateless applications — the price of admission

The problem: request 1 (login) hits app-1, which stores the session in ITS memory. Request 2 hits app-2 → "who are you?" — random logouts, lost carts.

Stateless = the app instance keeps NO client state between requests; any instance can serve any request. All state is externalized:

State in the instance ❌	Externalized ✅
HTTP session in JVM memory	**Redis** (Spring Session — a dependency + 2 config lines)
or session at all	**JWT** — state lives in the signed token (Phase 3)
uploaded files on local disk	**S3** (Phase 8)
in-memory caches per node	Redis shared cache
scheduled jobs on "the" server	one elected runner / separate worker / distributed lock (ShedLock)

(Sticky sessions — LB pins each user to one server — is the crutch: breaks scaling balance and loses sessions when a node dies. Know it, avoid it.)

★ Interview one-liner: "*Statelessness is what turns 'my server' into 'any server'.*"

5. Caching strategies — the highest-leverage performance tool

Reading from memory ≈ 1000× faster than a DB query. Layers (each shields the next):

```
browser cache → CDN (Phase 8) → NGINX proxy cache (Phase 5) → app cache (Redis) → DB
```

App-level patterns

- **Cache-aside** (standard): read → Redis hit? return : query DB → store in Redis with TTL → return. Spring: `@Cacheable("products")`.
- **Write-through / write-behind**: update cache on writes (rarer, more complexity).

The two hard problems

1. **Invalidation** — stale data. Tools: TTLs (honest staleness budget: "products may be 60 s old"), explicit eviction on update (`@CacheEvict`), event-driven invalidation.
2. **Stampede** — hot key expires → 10,000 requests hit the DB simultaneously. Tools: lock/single-flight recomputation, jittered TTLs, refresh-ahead.

Cache what is: read-often, written-rarely, tolerant of slight staleness (product catalog: yes; account balance: careful; auth decisions: no).

6. Database scaling — the hardest layer

Order of operations (do NOT jump ahead — each step is 10× cheaper than the next):

1. **Fix the queries**: indexes for real access patterns (`EXPLAIN ANALYZE` !), kill N+1s (JPA's favorite crime), select only needed columns, pagination.
2. **Connection pooling**: HikariCP sized sanely (pool of ~10–20 beats 200; DB connections are expensive). Many app instances × pool size must fit the DB's max_connections!
3. **Cache** (§5) — remove reads from the DB entirely.
4. **Vertical**: bigger RDS instance. Boring, effective.
5. **Read replicas** ↓
6. **Sharding** — split data across DBs by key (user 1–1M → shard A…). Enormous complexity (cross-shard queries, rebalancing). For most companies: never needed. Know the concept, resist the urge.

Read replicas

Most apps read 10–100× more than they write. Replicas take the read traffic:

```
writes ──► PRIMARY ──(async replication)──► REPLICA 1, REPLICA 2
reads  ──────────────────────────────────►  (round-robin)
```

- **Replication lag** (ms–seconds, async): read-your-own-writes problem — user saves profile (primary), next GET reads a replica that hasn't caught up → "my edit vanished!" Fixes: read-after-write goes to primary (session pinning), or tolerate it in UX.
- Spring: routing DataSource (`@Transactional(readOnly=true)` → replica) or app-level split.
- Replicas ≠ backup, ≠ HA of the primary. Which leads to…

7. High availability (HA)

Availability is measured in nines: 99.9% = 8.8 h down/year; 99.99% = 53 min. Each nine multiplies cost/complexity — pick what the business needs.

Principle: **no single point of failure (SPOF)** — everything critical exists ≥2×, with automatic failover:

- App tier: ≥2 instances behind LB, **in different AZs** (Phase 8), LB health checks eject dead nodes.
- DB: RDS Multi-AZ — synchronous standby, auto-failover ~1–2 min (vs replicas: async, for reads).
- Redis: replica + Sentinel, or managed (ElastiCache).
- LB itself: managed ALB is already multi-AZ (self-hosted NGINX LB needs its own pair + failover IP).
- **Hunt SPOFs by walking the request path from Phase 1**: DNS → CDN → LB → app → DB → cache. Anything that exists once, fails once.

Health checks make HA real: `/actuator/health` checked every 5–10 s; fail 3× → stop routing there (and Phase 11: restart it).

8. The standard scaling story (memorize as a narrative)

```
1 server (app+db together)                    ← Phase 7. Fine! Most apps live here.
→ split DB to its own machine/RDS
→ vertical scaling + indexes + Redis cache    ← cheap wins, in that order
→ stateless app + 2-3 instances + LB (+ multi-AZ)   ← availability & capacity
→ read replicas; CDN for assets
→ auto scaling on metrics
→ (only with extreme scale/org size): sharding, microservices, event-driven
```

Every arrow is triggered by a *measured* bottleneck, not by fashion. Instagram served millions with a handful of engineers on roughly this stack.

9. Common mistakes

- Premature complexity: microservices/K8s/sharding for 100 users ("resume-driven development").
- Scaling out an app that stores sessions/files locally (random logouts, vanished uploads) — statelessness first!
- Caching without TTLs or invalidation plan; caching everything, then debugging staleness for a week.
- Adding replicas before adding indexes (scaling a slow query = many machines running the slow query).
- Forgetting connection math: 10 instances × 50 pool = 500 connections → RDS max 100 → outage.
- "HA" with two app servers… in the same rack, one DB, one LB box (SPOFs everywhere).

10. Interview questions

1. Vertical vs horizontal scaling — trade-offs, and what does horizontal REQUIRE of the app?

2. What is a stateless app? Where do sessions/files/caches go? (Redis/JWT/S3 — with trade-offs.)
3. Explain cache-aside with TTL. What are invalidation and stampede, and defenses for each?
4. Read replicas: how do they work, what is replication lag, what is read-your-own-writes and how do you fix it?
5. Multi-AZ (HA) vs read replica (scaling) — the classic RDS interview distinction.
6. Order the database scaling playbook and justify the order.
7. Design 99.9% availability for a Spring Boot + PostgreSQL app — walk the path, eliminate every SPOF.
8. Your p95 latency doubled at 2× traffic. Walk your diagnosis (metrics → which layer saturates → matching remedy).

11. LAB — see it break, then fix it (local, Docker)

```
# 1. break statefulness visibly
docker compose up -d --scale api=3          # 3 app instances (NGINX round-robins — Phase 10 preview)
# login (session in JVM) → refresh repeatedly → randomly logged out! (watch which container answers:
#   add an /instance endpoint returning HOSTNAME)

# 2. fix with Spring Session + Redis (2 deps + spring.session.store-type=redis)
# → refresh: session survives across all 3 instances

# 3. cache-aside
# add @Cacheable to a product-list endpoint; benchmark:
sudo apt install apache2-utils
ab -n 2000 -c 50 http://localhost/api/products     # before vs after — record p95!

# 4. replication lag, simulated: read the same key from cache while updating DB without eviction
#    → observe staleness; add @CacheEvict; observe fix
```

12. ASSIGNMENT 09 (submit to Claude)

1. Lab results: paste the /instance outputs proving round-robin, the session bug, and the fix; the `ab` numbers before/after caching.
2. Your company's monolith (Spring Boot, 1 VPS, sessions in memory, files on disk, PostgreSQL same box) must reach 10× traffic and 99.9% availability in 3 months. Write the migration plan as ordered steps, each with: what changes, why now, what breaks if skipped.
3. Design the caching strategy for an e-commerce site: product pages, prices, stock levels, cart, user profile. For each: cache or not, where, TTL, invalidation — justify.
4. Explain read-your-own-writes to a junior with a concrete Spring Boot + replica scenario and two fixes.
5. Spot the SPOFs: `users → 1 NGINX VPS → 3 app containers on that same VPS → RDS Single-AZ → 1 Redis container`. List every SPOF and the fix for each.